

Inteligencia artificial avanzada para la ciencia de datos

Oswaldo Isaias Hernández Santes
Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias
Santiago de Querétaro, Querétaro; México
a01199004@tec.mx

Abstract—Este proyecto implementa un modelo de regresión lineal para predecir la edad de abalones (medida en número de anillos) a partir de sus medidas físicas, utilizando el conjunto de datos Abalone del UCI Machine Learning Repository. Se desarrollaron dos implementaciones una manual con gradiente descendente sin el uso de framework y otra con el uso de framework scikit-learn usando el SGDRegressor. Ambos modelos fueron evaluados con conjuntos de entrenamiento, validación y prueba en un 70%/15%/15% respectivamente monitoreando el error cuadrático medio (Mean Square Error) a lo largo de las epochs para diagnosticar el ajuste del modelo. Los resultados muestran que ambas implementaciones presentan un sesgo alto y una varianza baja, es decir, underfitting, el modelo sin framework alcanzó un MSE de prueba de 6.84 con R^2 de 0.34 mientras que el modelo con framework gracias a las actualizaciones de pesos por instancia de SGDRegressor mejoró a un MSE de prueba de 4.69 con un R^2 de 0.57. Como complemento se aplicó regularización L2 (Ridge) sobre el modelo con framework confirmando que esta técnica no resuelve el problema ya que está diseñada para combatir el sobreajuste y no el subajuste, además se exploró el efecto de ajustar el hiperparámetro de tasa de aprendizaje. Este proyecto ofrece una comparación práctica entre una implementación manual y una basada en framework de un mismo algoritmo y evidencia la importancia de diagnosticar correctamente el tipo de error de un modelo antes de aplicar técnicas de mejora.

Keywords—Regresión Lineal, Gradiente Descendente, Análisis Exploratorio de Datos (EDA), Machine Learning (ML), Inteligencia Artificial (IA), Split Test, Épocas/EPOCHS, Learning Rate, Parámetros, Hiperparámetros, Python, Instancias, Features, Error Cuadrático Medio (MSE), Coeficiente de Determinación (R^2), Frameworks, Scikit-learn, SGDRegressor, Regularización L2/Ridge, Sesgo, Varianza, Underfitting/Overfitting, One-Hot Encoding, Estandarización Z-score

I. INTRODUCCIÓN

La inteligencia artificial (IA) ha evolucionado significativamente desde sus inicios a mediados del siglo XX [1]. Originalmente la IA se centraba en la creación de sistemas capaces de realizar tareas que normalmente requerirían esfuerzo humano, pero con el tiempo se ha transformado en varios subcampos siendo el aprendizaje automático o machine learning (ML) uno de los más importantes. ML se refiere a la capacidad de las máquinas para aprender de datos y mejorar su rendimiento sin ser programadas explícitamente para cada tarea [2].

Dentro del aprendizaje supervisado que involucra entrenar un modelo con datos etiquetados existen dos grandes tipos de tareas clasificación en donde se predice una categoría, y regresión en donde se predice un valor numérico continuo [3]. Este proyecto se enfoca en un problema de regresión y se busca predecir la edad de un abalón (un molusco marino) a partir de sus medidas físicas, usando un algoritmo de regresión lineal entrenado mediante gradiente descendente.

A. Contexto general

Determinar la edad de un abalón de manera tradicional implica cortar su concha, teñirla y contar sus anillos de crecimiento bajo un microscopio el cual es un proceso lento y tardado. El objetivo de este proyecto es entrenar un modelo que estime esta edad a partir de medidas físicas evitando el proceso manual.

Para este proyecto se implementaron dos versiones del mismo algoritmo de regresión lineal una manual programando el gradiente descendente desde cero y otra usando el framework scikit-learn específicamente el estimador SGDRegressor el cual aplica descenso de gradiente estocástico. Comparar ambas implementaciones permite entender tanto el funcionamiento interno del algoritmo como las ventajas de usar un framework optimizado.

B. Base de Datos

El conjunto de datos utilizado proviene del UCI Machine Learning Repository [4], y contiene 4,176 instancias de abalones descritos por 8 atributos los cuales son sexo, longitud, diámetro, altura, peso total, peso de la carne (shucked), peso de la víscera, peso de la concha, y el número de anillos que se usa para estimar la edad sumándole 1.5 [4]. La Tabla 1 resume estas variables.

TABLE I. VARIABLES DEL DATASET ABALONE

Variable	Descripción	Unidad
Sex	Sexo del abalón: M (macho), F (hembra), I (infante)	—
Length	Medida más larga de la concha	mm
Diameter	Medida perpendicular a Length	mm
Height	Altura con la carne dentro de la concha	mm
Whole weight	Peso total del abalón	g
Shucked weight	Peso de la carne (sin concha)	g
Viscera weight	Peso de la víscera (tras desangrado)	g
Shell weight	Peso de la concha seca	g
Rings (Class)	Número de anillos; +1.5 = edad en años	—

II. ANÁLISIS EXPLORATORIO DE DATOS

A. Limpieza de datos

Al revisar el dataset se encontraron 2 instancias con un valor de 0 en la variable Height lo cual es físicamente imposible para un abalón. Dado que ambas correspondían a ejemplares de sexo infante (I) y representaban una fracción mínima de las 1,342 instancias de ese sexo, se decidió eliminarlas del conjunto de datos, quedando 4,174 instancias válidas.

B. Correlación de variables

Se calculó la matriz de correlación entre los atributos numéricos y la variable Class (número de anillos) para entender qué variables tienen mayor poder predictivo. El peso de la concha (Shell weight) presentó la correlación más alta aproximadamente de 0.628, seguido del diámetro de aproximadamente 0.575, la altura de aproximadamente 0.558 y la longitud de aproximadamente 0.557. El peso completo de aproximadamente 0.541, el peso de la víscera de aproximadamente 0.504 y el peso de la carne de aproximadamente 0.421 mostraron correlaciones más bajas, pero se conservaron para que el modelo contemplara la proporción entre masa interna y externa del abalón.

C. Transformación de los datos

La variable Sex es categórica (M, F, I) por lo que se transformó usando One-Hot Encoding [5], generando las columnas Sex_M, Sex_F y Sex_I, pero en la versión final del modelo únicamente se utilizaron las 7 variables numéricas continuas (longitud, diámetro, altura y los cuatro pesos) mientras que las columnas de sexo no se incluyeron en el vector de entrada X, esta es una limitación conocida del modelo ya que el sexo del abalón sí mostró cierta correlación con su edad.

D. Estandarización

Antes de entrenar el modelo, las 7 variables numéricas se estandarizaron utilizando la técnica de Z-score [6], que consiste en restar la media y dividir entre la desviación estándar de cada atributo de esta forma cada variable queda con media 0 y desviación estándar 1, esto es importante porque las variables del dataset tienen magnitudes muy distintas (por ejemplo, el peso completo llega hasta 2.83, mientras que la altura no supera 1.13) y sin este paso el algoritmo de gradiente descendente podría interpretar erróneamente que los atributos con valores más grandes son más importantes.

III. MODELO

A. Justificación

Se utilizó un modelo de regresión lineal, ya que la variable a predecir (el número de anillos/edad) es continua [3], [7]. La función de hipótesis del modelo es:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_7x_7 + b \quad (1)$$

donde X son los siete atributos estandarizados, W son los pesos que el modelo ajusta durante el entrenamiento y b es el sesgo que representa la edad base estimada cuando todos los atributos valen su media o 0 después de la estandarización.

Como función de costo se utilizó el Error Cuadrático Medio (Mean Square Error) [7]:

$$MSE = (1/n) \sum (\hat{y}_i - y_i)^2 \quad (2)$$

que calcula el promedio de las diferencias al cuadrado entre las predicciones y los valores reales penalizando más los errores grandes. Para minimizar esta función de costo se utilizó el algoritmo de gradiente descendente [8], que ajusta iterativamente los pesos y el sesgo en dirección opuesta al gradiente de la función de costo:

$$dw = (1/n) \cdot X^T(\hat{y} - y), \quad db = (1/n) \cdot \sum(\hat{y} - y) \quad (3)$$

$$w = w - \alpha \cdot dw, \quad b = b - \alpha \cdot db \quad (4)$$

donde α es la tasa de aprendizaje (learning rate), que controla el tamaño de cada paso de ajuste.

B. Preparación de los datos

El dataset se dividió en tres subconjuntos uno de entrenamiento (70%), validación (15%) y prueba (15%) [9]. En la implementación manual esta separación se realizó permutando aleatoriamente los índices del dataset con una semilla fija de 42 y cortando el arreglo en las proporciones antes mencionadas que son 2,921 instancias de entrenamiento, 626 de validación y 627 de prueba.

En la implementación con framework se utilizó la función train_test_split de scikit-learn con la misma semilla, en dos pasos consecutivos donde primero se dividió en 70%/30% y luego 50%/50% sobre el 30% para tener 15%/15%, obteniendo los mismos tamaños de conjunto. El conjunto de validación permite monitorear el modelo durante el entrenamiento para detectar problemas de ajuste, mientras que el conjunto de prueba se reserva para la evaluación final [9].

C. Hiperparámetros iniciales

Ambas implementaciones se entrenaron con los mismos hiperparámetros iniciales para poder comparar de forma igual la tasa de aprendizaje (α) fue de 0.01 usando 1,000 épocas y los pesos junto con el sesgo inicializados en cero.

IV. RESULTADOS — MODELO SIN FRAMEWORK

El modelo implementado manualmente con gradiente descendente fue entrenado durante 1,000 épocas. La Fig. 1 muestra la curva de aprendizaje donde el error inicial fue de aproximadamente 109 y aproximadamente en la época 300 el modelo se estabiliza en un MSE de aproximadamente 5.17 para entrenamiento, 4.71 para validación y 6.84 para prueba.

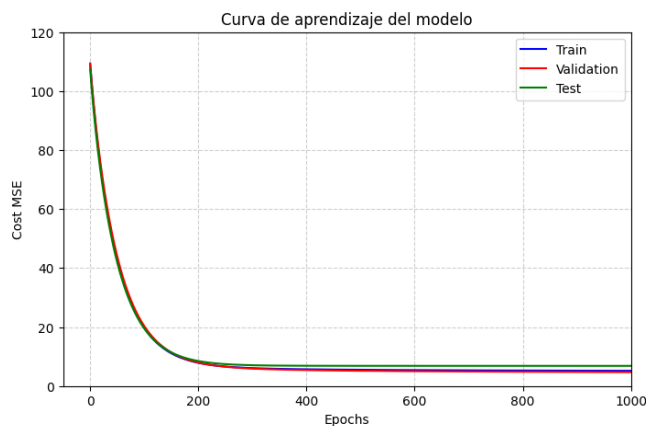


Fig. 1. Curva de aprendizaje del modelo sin framework (train, validation, test).

El sesgo (bias) obtenido fue de 9.9328 el cual representa la edad base estimada de un abalón con atributos promedio. Los pesos con mayor magnitud fueron Shucked weight (la feature 5 aproximadamente de -1.7897) y Shell weight (la feature 7 aproximadamente de 1.7488), es decir que por cada desviación estándar que aumenta Shucked weight, el modelo resta aproximadamente 1.79 años a la predicción, mientras que un aumento equivalente en Shell weight suma 1.75 años.

Las métricas finales en el conjunto de prueba fueron un MSE de 6.84 y un R^2 de 0.34 el cual equivale a un error promedio de aproximadamente 2.61 años (raíz del MSE) y a que el modelo únicamente logra explicar el 33% de la variabilidad de los datos. La Fig. 2 compara las predicciones del modelo contra los valores reales de edad en el conjunto de prueba y se observa una dispersión considerable de puntos alrededor de la línea de predicción perfecta.

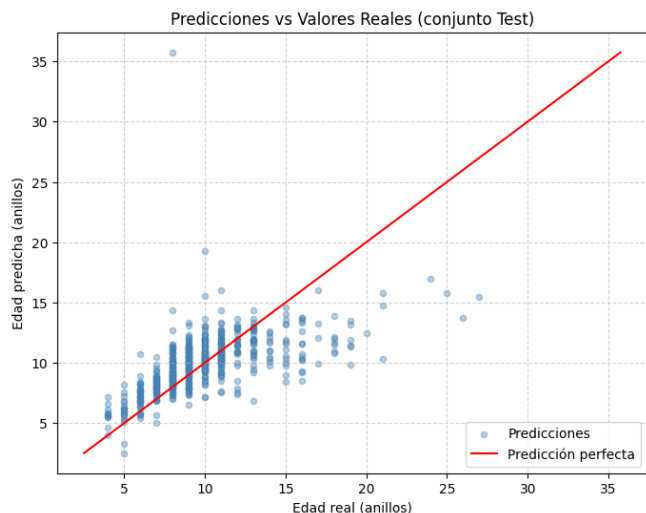


Fig. 2. Predicciones vs valores reales del modelo sin framework (conjunto de prueba).

Tanto la baja diferencia entre los errores de entrenamiento, validación y prueba (varianza baja) como el nivel elevado de estos tres errores (sesgo alto) indican que el modelo presenta underfitting [12] y quiere decir que el modelo es demasiado simple para la complejidad del dataset.

V. MODELO CON FRAMEWORK (SGDREGRESSOR)

A. Justificación

Para la segunda implementación se utilizó el estimador SGDRegressor de scikit-learn [10], configurado con una tasa de aprendizaje constante (`learning_rate='constant'`), `eta0=0.01` y sin penalización (`penalty=None`) para tener las mismas condiciones que la implementación manual y a diferencia del gradiente descendente por lotes implementado manualmente — que actualiza los pesos una sola vez por época, usando todo el conjunto de entrenamiento — el SGDRegressor mediante el método `partial_fit()` actualiza los pesos una vez por cada instancia que procesa [10], esto quiere decir que, en 1,000 épocas sobre 2,921 instancias de entrenamiento el modelo con framework realiza aproximadamente 2.9 millones de actualizaciones de pesos comparado con las 1,000 actualizaciones del modelo manual.

B. Resultados

La Fig. 3 muestra la curva de aprendizaje del modelo con framework donde el error converge casi de inmediato (desde la época 1-2) estabilizándose en un MSE de aproximadamente 4.99 para entrenamiento, 4.90 para validación y 4.69 para prueba.

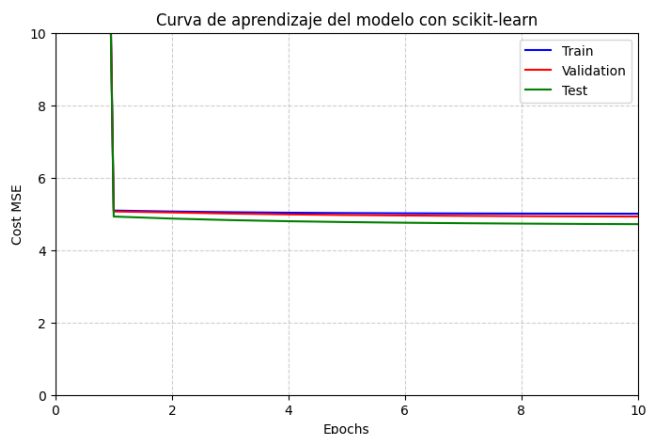


Fig. 3. Curva de aprendizaje del modelo con framework (SGDRegressor).

Los coeficientes obtenidos cambiaron considerablemente respecto al modelo manual donde el sesgo fue de aproximadamente 9.9269 y ahora los pesos con mayor impacto son Whole weight (feature 4 con aproximadamente 3.1704), Shucked weight (feature 5 con aproximadamente 4.0716), Diameter (feature 2 con aproximadamente 1.0973) y Shell weight (feature 7 con aproximadamente 1.8252) este cambio en la magnitud de los coeficientes puede explicarse por la gran diferencia en el número de actualizaciones de pesos entre ambas implementaciones.

En el conjunto de prueba el modelo con framework obtuvo un MSE de 4.6901 y un R^2 de 0.5689 el cual nos da un error promedio de 2.1657 años y esto representa una mejora notable respecto al modelo manual (MSE de 6.84 y R^2 de 0.34) el modelo con framework explica el 57% de la variabilidad de los datos comparado con el 34% del modelo manual. La Fig. 4 muestra que las predicciones del conjunto de prueba se agrupan

más cerca de la línea de predicción perfecta que en el modelo manual.

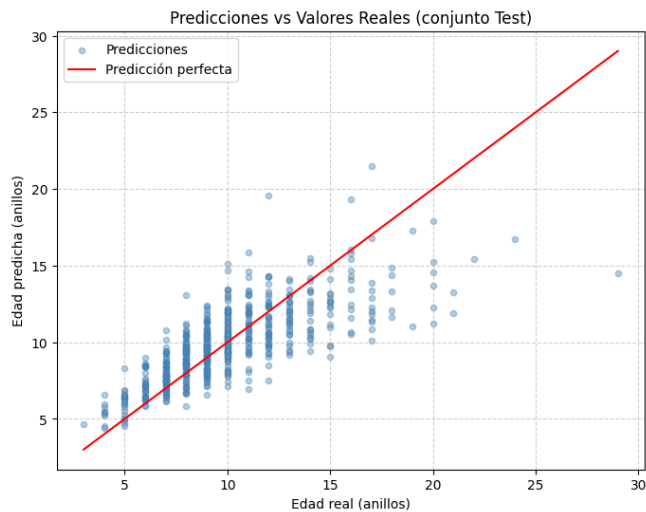


Fig. 4. Predicciones vs valores reales del modelo con framework (conjunto de prueba).

A pesar de la mejora el modelo con framework sigue presentando una varianza baja y un sesgo alto, por lo que el diagnóstico de underfitting [12] sigue igual. En resumen el modelo aunque está mejor ajustado que la implementación manual sigue siendo demasiado simple para el dataset.

VI. REGULARIZACIÓN L2 (RIDGE)

La regularización L2, también conocida como regresión Ridge es una técnica que añade a la función de costo una penalización proporcional al cuadrado de los pesos del modelo [11]:

$$\text{Costo} = \text{MSE} + \lambda \cdot \sum w^2 \quad (5)$$

donde λ (lambda) controla la intensidad de la penalización. Esta técnica reduce la magnitud de los pesos del modelo para evitar que se ajuste demasiado a los datos de entrenamiento y se usa normalmente para reducir el sobreajuste (overfitting) [11], [12].

Debido a que ambos modelos de este proyecto presentan underfitting y no overfitting se esperaba que la regularización L2 no mejorara de alguna forma al modelo, aún así se aplicó sobre el modelo con framework esto con el objetivo de comprobar esta hipótesis además de dar un ejemplo del uso de esta técnica.

Usando el estimador Ridge de scikit-learn con $\lambda=0.1$ se obtuvo un MSE prácticamente igual al modelo sin regularización tanto en entrenamiento como en validación y prueba, esto confirma que la penalización no tiene ningún efecto cuando el problema de fondo es que el modelo es demasiado simple.

Para profundizar en este comportamiento, se entrenaron modelos Ridge con distintos valores de λ (0.001, 0.01, 0.1, 1, 10 y 100). La Fig. 5 muestra que, mientras λ se mantiene pequeño (≤ 1), el MSE no cambia, pero este al aumentar

considerablemente la fuerza de la regularización ($\lambda \geq 10$) el MSE en los tres conjuntos empieza a crecer, es decir, el underfitting aumenta y esto nos confirma que restringir aún más los pesos de un modelo que ya es demasiado simple únicamente empeora las predicciones y confirma que la regularización L2 no es la técnica adecuada para este caso.

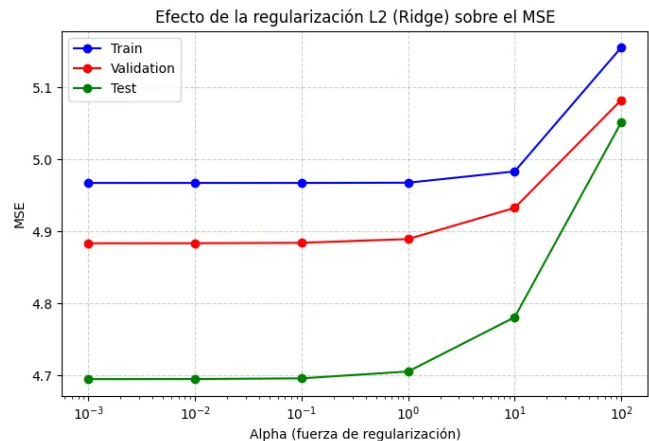


Fig. 5. Efecto de la regularización L2 (Ridge) sobre el MSE al variar α (λ).

VII. AJUSTE DE HIPERPARÁMETROS

Como segundo intento de mejora sobre el modelo con framework se ajustó el hiperparámetro de tasa de aprendizaje (learning rate) de SGDRegressor esto entrenando el modelo con distintos valores (0.001, 0.005, 0.01, 0.05 y 0.1) y comparando el MSE resultante en los tres conjuntos de datos.

Los resultados mostraron que el MSE no varía de forma significativa dentro de este rango de valores y muestra una ligera tendencia de mejora al aumentar la tasa de aprendizaje pero no tenemos cambios significativos en el R^2 y esto quiere decir que debido al problema del modelo el cual es la simplicidad para el dataset que se está usando ajustar este hiperparámetro específico tiene un margen de mejora limitado y sería necesario buscar otros cambios que puedan mejorar el modelo ya sea agregar variables como el sexo del abalón o introducir hacer más complejo el modelo para reducir el sesgo del modelo de forma más significativa y que pueda predecir de manera más adecuada la edad del abalón.

VIII. CONCLUSIONES

En este proyecto se implementó un modelo de regresión lineal para predecir la edad de abalones tanto de forma manual con gradiente descendente como utilizando el framework scikit-learn (SGDRegressor).

Los resultados muestran de manera consistente que ambos modelos presentan underfitting evidenciado por errores considerables en los tres conjuntos de datos y una varianza baja ya que los errores de entrenamiento, validación y prueba se comportan de forma muy similar entre sí.

El modelo con framework superó de forma clara al modelo manual con un MSE de prueba de 4.69 frente a uno de 6.84 y una R^2 de 0.57 frente a una de 0.34 esto se explica principalmente por la forma en que SGDRegressor actualiza los pesos.

También se comprobó que el uso de regularización L2 no mejora de ninguna forma el modelo ya que el problema real del modelo es underfitting en lugar de overfitting y los pesos ya son relativamente pequeños y penalizarlos aún más solamente empeora el ajuste. De igual forma el ajuste del hiperparámetro de tasa de aprendizaje tuvo un impacto ligero sobre el desempeño del modelo.

Estos resultados apuntan a que la causa principal del underfitting no es un problema de optimización, se debe más a la complejidad del modelo donde se hace una combinación lineal de siete variables físicas y es insuficiente para capturar toda la complejidad de la relación entre las medidas de un abalón y su edad. Como trabajo futuro, se podría explorar la inclusión de la variable de sexo así como el uso de términos polinomiales o de un modelo no lineal ya sea un árbol de decisión o alguna posible mejora que permita predecir lo más exacto posible la edad de un abalón.

REFERENCIAS

- [1] Evolución de la Inteligencia Artificial. Iberdrola. <https://www.iberdrola.com/innovacion/evolucion-inteligencia-artificial>
- [2] ¿Qué es machine learning? SAP. <https://www.sap.com/latinamerica/products/artificial-intelligence/what-is-machine-learning.html>
- [3] D. Richardson, "La IA y el ML y su importancia para las empresas," Red Hat Blog, 2024. <https://www.redhat.com/es/blog/what-aiml-and-why-does-it-matter-your-business>
- [4] W. Nash, T. Sellers, S. Talbot, A. Cawthorn, and W. Ford, "Abalone," UCI Machine Learning Repository, 1994. <https://doi.org/10.24432/C55C7W>
- [5] "One-Hot Encoding: ¿Qué es? ¿Cómo se utiliza?," Liora, 28 enero 2026. <https://liora.io/es/one-hot-encoding-que-es>
- [6] "Datos numéricos: Normalización," Machine Learning Crash Course, Google for Developers. <https://developers.google.com/machine-learning/crash-course/numerical-data/normalization>
- [7] "La Regresión Lineal en el Machine Learning," Codificando Bits. <https://codificandobits.com/blog/regresion-lineal/>
- [8] "Descenso de gradiente para regresión lineal [Guía 2025]," KeepCoding, 2024. <https://keepcoding.io/blog/descenso-de-gradiente-para-regresion-lineal/>
- [9] "How To Choose The Right Test Options When Evaluating Machine Learning Algorithms," Machine Learning Mastery. <https://machinelearningmastery.com/how-to-choose-the-right-test-options-when-evaluating-machine-learning-algorithms/>
- [10] "SGDRegressor," scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDRegressor.html
- [11] "¿Qué es la regresión Ridge?," IBM. <https://www.ibm.com/es-es/think/topics/ridge-regression>
- [12] "ML | Underfitting and Overfitting," GeeksforGeeks. <https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>